# hydra_circuit_satsuma on the SAT Competition 2022 Anniversary Track: Where the Structural Stages Pay, and What They Cost

Greg Sidebottom

October 11, 2026

**Abstract**

`hydra_circuit_satsuma` is the SAT Competition 2026 winner, `satsuma-iter+kissat`, with four certified structural stages in front of it: a Cook prover for pigeonhole-type shapes, a numeric factoring stage for multiplier circuits, a GF(2) elimination stage for parity systems, and a circuit stage that re-encodes hidden multipliers. We ran it beside `satsuma` itself on all 5,355 instances of the SAT Competition 2022 anniversary track, the union of every application, crafted and main track since 2002, with a 5,000 s limit. `hydra_circuit_satsuma` solved 4,403 instances to `satsuma`'s 4,395, with a PAR-2 of 1,982.7 against 1,999.3, and eight instances only it solved. Both beat the 2022 field's virtual best of 4,345 and, after correcting for the Apple M4 Pro's measured 3.0-fold speed over the competition's StarExec nodes, its winner by a wide margin. The PAR-2 gap of 16.65 points decomposes exactly: 14.9 points from the eight unique wins, 2.7 from 256 instances the stages decide in milliseconds where `satsuma` searches, and 1.1 points paid as the stages' prefix on the 5,037 instances they hand on. Only 8 of the 264 instances the stages decide became unique wins because `satsuma`'s symmetry breaking refutes 207 of them by itself; the eight are the parity formulas wrapped in cardinality constraints and the two largest mutilated chessboards, where that breaking finds nothing. Nearly half of the cost was a bug, fixed after the run.

## 1 Introduction

The companion document `hydra_satsuma_solver` describes the solver and its results on the 2026 main track, where `hydra_circuit_satsuma` solves four more instances than `satsuma` out of 391. A 391-instance set with four unique wins says little about where the stages pay in general, so we ran the same two solvers over the 2022 anniversary track [1], the largest benchmark set the competition has published: 5,355 instances from 133 families, covering twenty years of application, crafted and main tracks. The run took seven days on ten cores. This document reports the result and answers the question the run was designed for: for each stage, on how many instances did it help, by how much, and what did it cost on the instances where it did not.

## 2 The two solvers

`satsuma` is `satsuma-iter+kissat` as submitted to the 2026 competition [2]: satsuma's symmetry preprocessing in its iterative mode, which detects symmetries with dejavu and adds units, binary and lex-leader clauses with a binary substitution-redundancy proof, followed by kissat, whose refutation is appended to the same proof. We run the official image unchanged. Our `sat` binary parses the formula, writes it to the container, runs the official script, and projects a model back to the original variables.

`hydra_circuit_satsuma` runs four stages on the parsed formula before handing it to that same engine, byte for byte:

1. the *Cook prover*: recognisers for pigeonhole, embedded pigeonhole, round-robin, clique-colouring and mutilated chessboard shapes, each refuted by a short cutting-planes argument written as a VeriPB proof [3, 4];
2. the *factoring stage*: multiplier recognition and numeric factoring of the output constant, which decides a factoring instance with the model as certificate;

3. the *XOR stage*: recovery of complete parity encodings and Gaussian elimination over GF(2); an inconsistent system is refuted with a Gocht–Nordström parity proof [5], a consistent pure-XOR formula gets its model from back-substitution [6, 7, 8];
4. the *circuit stage*: trial factoring of the gate structure to find hidden multipliers, and a certified re-encoding when enough are found; it fires on none of the 5,355 instances, so here it is pure cost.

Each stage either decides the instance with a certificate or hands the unchanged formula to the next; the engine sees the original formula. In this run certificate checking was off: the stage proofs were written, not checked. In the October 2026 main-track run with checking on, all 27 Cook and 11 XOR certificates verified under VeriPB, and `satsuma`'s own certificates are checked by `dsr-trim` there.

# 3 The benchmark and the protocol

**Instances.** All 5,355 instances of the anniversary track, in GBD's order (sorted by hash, which mixes the families), with GBD's recorded status for 4,715 of them. The two solvers' answers agree with every recorded status.

**Machine and limits.** An Apple M4 Pro (10 performance and 4 efficiency cores, 64 GB), ten jobs at a time so no job lands on an efficiency core. The engine runs in Docker Desktop with a 32 GB VM; a memory-aware scheduler admits instances by estimated footprint against a 26 GB VM budget and a 20 GB host budget. 5,000 s of wall-clock per instance and solver, PAR-2 scoring (an unsolved instance counts 10,000 s).

**Pairing.** `tools/gbd/duo.py` runs both solvers through the set in the same order and starts the two jobs of an instance together, so they meet the same machine load. The 54 instances that started before the switch to pairing, three hours in, and that no stage decided were run in full by both. From then on `hydra_circuit_satsuma` ran only its stages: when they hand the formula on, its result is `satsuma`'s on the identical formula, and its time is its own prefix plus `satsuma`'s engine time. This is exact rather than an approximation: the engine's input is byte-identical, and satsuma and kissat are deterministic on identical input, which we checked on the full pairs. It halved the compute and, more importantly for this document, gives the prefix of every hand-off as a measured number.

**Speed against 2022.** The anniversary track ran on StarExec's Intel Xeon E5-2609 nodes (2.4 GHz, 2012) and reports CPU seconds. To compare counts under the same 5,000 s limit we timed the 2022 track's most-solved solver, `kissat-sc2022-bulky`, built from its competition tag, on twenty anniversary instances spread over families and solve times, in the same Docker VM at the same ten-job load. Its search does not depend on time, so the ratio of its times is the hardware's: the median was 2.99, quartiles 2.79 to 3.12, range 1.82 to 3.83, with every verdict as recorded. We use 3.

**Timeline.** The run started 2026-10-04 14:23 on eight cores, switched to the paired-reuse protocol at 17:18, to ten cores at 22:30, and finished 2026-10-11 15:08, after seven days and 39 minutes of activity at a mean pace of about 32 instances per hour. Two events cost capacity and nothing else: half an hour at nine cores for the speed calibration, and 53 minutes on 10-06 during which a scheduler bug held the queue behind one job and three jobs ran instead of ten. A cheaper implementation of the circuit stage's check, with identical results, was installed after 164 instances.

# 4 Results

Table 1 gives the score. `hydra_circuit_satsuma` solves eight more instances than `satsuma` and loses none: every instance `satsuma` solves, `hydra_circuit_satsuma` solves, because the engine is the same and the stages only ever decide an instance or step aside. Figure 1 is the cactus plot. The eight instances only `hydra_circuit_satsuma` solved are in Table 2; all eight are decided in milliseconds by a stage, and `satsuma` ran the full 5,000 s on each. None of the 28 solvers of 2022 solved seven of them; mchess_20 was solved by seven.

| | solved | SAT | UNSAT | unsolved | PAR-2 |
|---|---|---|---|---|---|
| `satsuma` | 4,395 | 1,990 | 2,405 | 960 | 1,999.3 |
| `hydra_circuit_satsuma` | **4,403** | 1,991 | 2,412 | 952 | **1,982.7** |
| 2022 virtual best (28 solvers) | 4,345 | 2,140 | 2,205 | 1,010 | 2,108.5 |
| 2022 winner, Kissat_MAB_ESA | 4,028 | | | | 2,806.4 |
| 2022 most solved, kissat-sc2022-bulky | 4,037 | | | | 2,811.0 |

Table 1: All 5,355 instances. The 2022 rows are the competition's own times on StarExec, so their counts compare with ours only after the speed correction of Section 8. The virtual best takes the fastest of the 28 sequential entrants on each instance.

| instance | family | stage | `hydra_circuit_satsuma` | 2022 |
|---|---|---|---|---|
| pmg-14 | long-learned-clauses | XOR, parity proof | 6.0 ms | 0/28 |
| eulcbip-8 | long-learned-clauses | XOR, parity proof | 5.2 ms | 0/28 |
| mchess_20 | mutilated-chessboard | Cook, VeriPB | 1.9 ms | 7/28 |
| pmg-13 | long-learned-clauses | XOR, parity proof | 6.9 ms | 0/28 |
| mod2-rand3bip-sat-300-3 | tseitin-formulas | XOR, model | 0.6 ms | 0/28 |
| eulcbip-9 | long-learned-clauses | XOR, parity proof | 6.7 ms | 0/28 |
| eulcbip-7 | long-learned-clauses | XOR, parity proof | 4.2 ms | 0/28 |
| mchess_19 | mutilated-chessboard | Cook, VeriPB | 1.6 ms | 0/28 |

Table 2: The eight instances `hydra_circuit_satsuma` solved and `satsuma` did not, in run order. The last column is how many of the 28 solvers of 2022 solved the instance.

Beyond the uniques, the two arms differ on the 264 instances a stage decides and on nothing else. On the other 256 of those, `satsuma` also solves the instance and `hydra_circuit_satsuma` is faster every time: the stages' median time on the 264 is 4.6 ms and their maximum 229 ms, against `satsuma`'s median of 0.34 s on the XOR-decided instances, 0.36 s on the Cook-decided ones and 21 s on the factoring ones. The large differences are the instances `satsuma` solves slowly: mchess_17 in 4,179 s, the satisfiable random parity system mod2-rand3bip-sat-290-2 in 4,057 s, pmg-12 in 3,063 s, mod2-rand3bip-sat-270-3 in 585 s, the factoring instance 10142772393204023nc in 489 s, mchess_18 in 341 s. Everywhere else `hydra_circuit_satsuma` is `satsuma` plus a prefix. The geometric mean of the time ratio `hydra_circuit_satsuma`/`satsuma` over the 2,543 instances both solved in at least 5 s is 1.042: the prefix, which is at most a few seconds except where a stage misbehaves (Section 7).

# 5 Where the PAR-2 difference comes from

The PAR-2 gap is $1{,}999.3 - 1{,}982.7 = 16.65$ points, which is 89,150 s of the summed score over 5,355 instances. Since the two arms share the engine, every instance falls into one of four classes, and the sum of the per-instance differences over the classes is the gap exactly. Table 3 and Figure 2 give the decomposition, with each class split by stage.

Three things stand out.

**Unique wins dominate.** Each is worth $10{,}000/5{,}355 = 1.87$ points, and the eight are 14.9 of the 16.65. A stage that solves one instance `satsuma` cannot is worth more than the stages' total cost over the whole set, and worth more than all 256 speed wins together. The speed wins are the stages' everyday business, 256 instances decided in milliseconds, but `satsuma` already solves most of them in under a second, so they add 2.7 points.

**The cost is small, and most of it was avoidable.** The 5,037 hand-offs paid 6,080 s of prefix in total, a mean of 1.21 s and 1.14 points. 1,124 hand-offs paid more than a second and 73 more than ten. Section 7 traces the cost to two sources, a bug now fixed and the circuit trial.

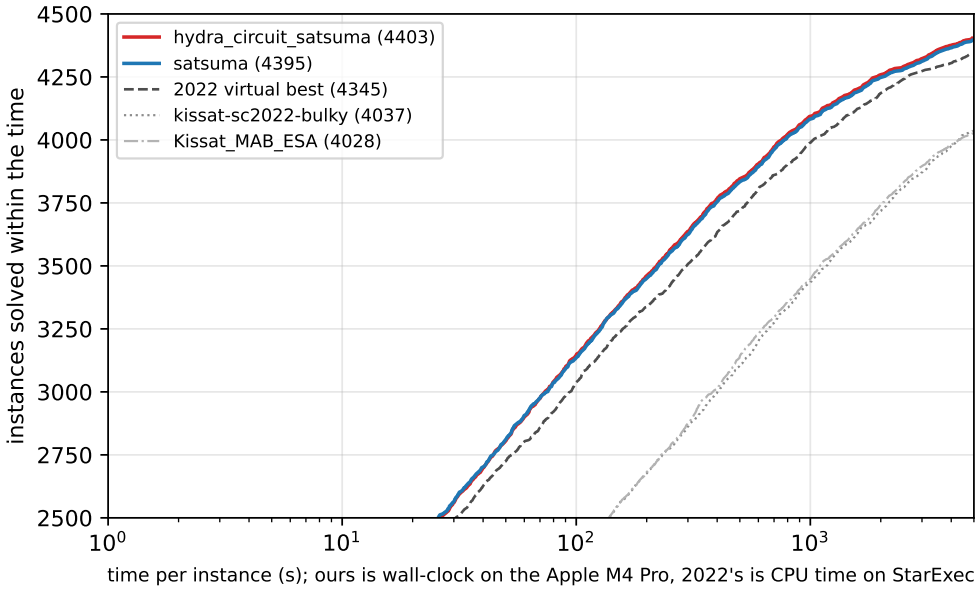



Figure 1: Instances solved within a per-instance time. Ours are wall-clock times on the Apple M4 Pro; the 2022 curves are CPU seconds on StarExec's Xeon E5-2609 nodes, about three times slower per job, so they sit to the right of where they would on the M4 Pro.

**The full-run pairs are noise.** On the 54 instances both arms ran in full, `hydra_circuit_satsuma` pays its prefix and then runs the same deterministic search; the net is −64 s, 43 of the 54 slightly in `satsuma`'s favour. Their spread, a few seconds per instance, is the measurement noise of two jobs sharing the M4 Pro with eight others.

## 6 Why only eight of 264

The stages decide 264 instances: 167 by the XOR stage (155 refutations, 12 satisfiable pure-XOR systems), 84 by the Cook prover and 13 by factoring. Of these, 129 were solved by none of the 28 solvers of 2022, yet only 8 became unique wins. The reason is `satsuma`'s symmetry breaking, and we can measure it: we ran satsuma's symmetry step alone, with the competition's flags and without kissat, on all 264 (`tools/gbd/sym_probe.py`).

Table 4 gives the result. On 207 of the 264 the symmetry step alone derives the empty clause, and `satsuma` solved every one, in a median of 0.33 s. These are the Tseitin, Urquhart and xor-chain formulas written as plain XOR clauses, and the pigeonhole and clique-colouring families: in a plain Tseitin encoding every variable sits in exactly two parity constraints, so flipping the edges around any cycle of the graph is a symmetry, and breaking those symmetries is a refutation. Our XOR stage refutes the same instances in milliseconds, but `satsuma` refutes them in a second, so they are speed wins.

The unique wins are where that mechanism breaks:

- **pmg** and **eulcbip** are Tseitin formulas inside cardinality constraints: exactly one of three edges at each vertex in pmg, exactly two of four in eulcbip, with one twisted pair making the total parity odd. Each vertex's clauses contain a complete XOR encoding, which the XOR stage reads, plus extra clauses that destroy the cycle flips. satsuma finds no symmetry on pmg-13 and pmg-14 and fixes one variable on the eulcbip instances, so kissat meets a parity formula at full strength, which resolution cannot refute in polynomial size [7]. pmg-12, the smallest, was the exception: 63 binary symmetry clauses were enough for kissat to finish, in 3,063 s.
- **mchess_19** and **mchess_20** are the largest mutilated chessboards; satsuma fixes two variables on each. `satsuma` solved mchess_15 to 18 in 84 to 4,179 s, in the 2022 field's difficulty order, and ran out of time on 19 and 20. The Cook prover counts squares of two colours [3].

| class | stage | instances | seconds | PAR-2 points |
|---|---|---|---|---|
| unique wins | XOR | 6 | +60,000 | +11.20 |
| | Cook | 2 | +20,000 | +3.73 |
| speed wins (both solve, the stage decides) | XOR | 161 | +8,715 | +1.63 |
| | Cook | 82 | +4,909 | +0.92 |
| | factoring | 13 | +835 | +0.16 |
| full-run pairs (before pairing) | | 54 | −64 | −0.01 |
| prefix on hand-offs | factoring trial | 5,037 | −73 | −0.01 |
| | XOR stage | 5,037 | −2,827 | −0.53 |
| | circuit trial | 4,866 | −2,997 | −0.56 |
| | other | 5,037 | −183 | −0.03 |
| total | | 5,355 | +89,150 | +16.65 |

Table 3: The PAR-2 gap decomposed. Seconds are the summed difference score(`satsuma`) − score(`hydra_circuit_satsuma`) over the instances of the row; points divide by 5,355. A unique win is worth 10,000 s minus the stage's milliseconds. The circuit trial runs on the 4,866 hand-offs with at most 10 million clauses; the 171 larger ones skip it. "Other" is the hand-off time the per-stage lines do not account for: the Cook recognisers on a miss, XOR recovery when it finds nothing, and clause copies.

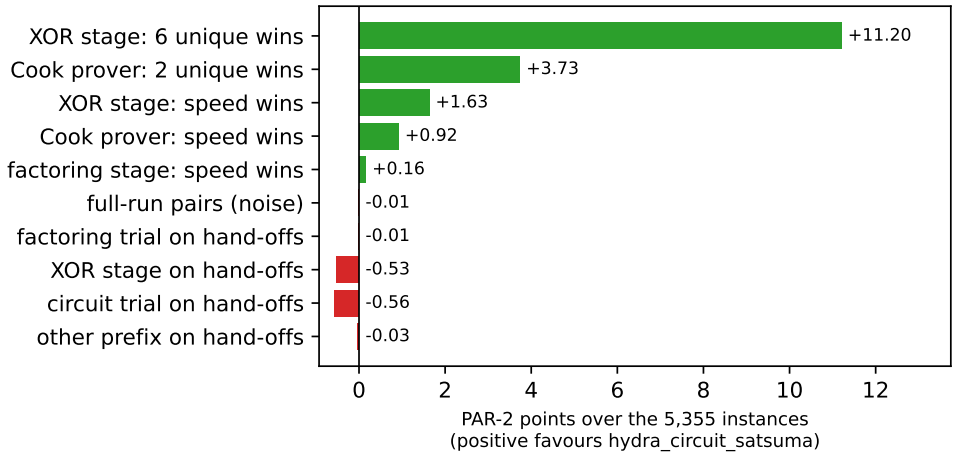



Figure 2: The rows of Table 3.

- **mod2-rand3bip-sat-300-3** is a satisfiable random parity system with every variable in three equations, so no flips exist and the outcome is search luck: `satsuma` solved the ten smaller members, in 3.9 s to 4,057 s, eight of them slower than the 2022 field's best time, and timed out on this one, which two of 28 solvers solved in 2022.

The lesson for future stages is a measurement, not a family list: a stage is worth building where the engine's own preprocessing has no purchase, and satsuma's symmetry step alone, run on candidate instances, tells which those are before any engine time is spent.

## 7 The cost, and where it went

The prefix of a hand-off is parse-free: the clock starts after parsing, which both arms do identically. It is the four stages' work on a formula they then hand on.

**The XOR stage, 2,827 s.** On a mixed formula whose parity system is consistent, the stage propagated the elimination's forced units through the whole formula with a fixpoint loop over all clauses, to build a simplified

| symmetry step alone | $n$ | solved | median | max | failed |
|---|---|---|---|---|---|
| derives the empty clause | 207 | 207 | 0.33 s | 13.8 s | none |
| adds clauses, no refutation | 30 | 25 | 11.9 s | 4,180 s | eulcbip-7, -8, -9; mchess_19, _20 |
| finds no symmetry | 27 | 24 | 24.1 s | 4,057 s | pmg-13, -14; mod2-rand3bip-sat-300-3 |

Table 4: The 264 stage-decided instances by what satsuma's symmetry step does on its own, with `satsuma`'s result in the run. The probe was made at instance 660 of the run and predicted the remaining unique wins: the six it rated strong all landed, one of its two likely candidates did (pmg-12 was solved in 3,063 s), and none of its low ones.

residual, and then discarded the residual because in certified mode the engine must refute the original formula. On the bitvector family the "system" is a single recovered unit clause and the loop cascaded it through two to three million clauses: 201 s on bvult_16967, 199 s on bvslt_19860, 110 to 136 s on the bvadd instances, 40 to 67 s on shift1add. That discarded propagation is 2,637 s of the stage's 2,827 s and 0.49 of the gap's points; kissat does the same propagation in under a second. It is fixed: the stage now returns right after the elimination when the residual cannot be used (`solve_xor_system(.., simplify_mixed=false)`), which brings bvult_16967's hand-off from 202.8 s to 2.6 s with identical verdicts, and would have made the gap 17.1 points. The bitvector family is the only one where `hydra_circuit_satsuma` lost ground, −1,608 s over 216 instances.

**The circuit trial, 2,997 s.** The circuit stage's trial factoring runs on every hand-off with at most ten million clauses and found a multiplier worth rewriting on none; the anniversary set predates the GenMul encodings it was built for, which entered the competitions in 2025. Its cost is spread thin, 0.6 s per instance, with the largest on the core-based-generator and planning instances with millions of gates (ncc_none_7047, 42 s; two more ncc instances, 24 s each; atco_enc3, 22 s). It is the one remaining cost worth gating, by the gate count or by stopping the trial early when the first products do not appear; at 0.56 points it is a third of a unique win.

**Everything else, 256 s.** The factoring stage's trial (73 s over 5,037 instances), the Cook recognisers and XOR recovery on misses, and the clause copies between stages cost 0.05 points together. The Cook prover, whose two unique wins and 82 speed wins are worth 4.65 points, costs nothing measurable.

# 8 Against the 2022 field

Both arms solved more instances than the 2022 virtual best, 4,403 and 4,395 against 4,345, on an M4 Pro three times faster per job than the competition's Xeon E5-2609 nodes. Scaling our times by a factor $k$ and recounting under the 5,000 s limit says how much of that is the hardware. `satsuma` keeps more solved instances than the 2022 winner Kissat_MAB_ESA while $k \leq 5.85$ and a lower PAR-2 while $k \leq 5.46$; `hydra_circuit_satsuma` while $k \leq 6.13$ and 5.64. At the measured $k = 3$ the counts are 4,201 and 4,212 with PAR-2 2,461 and 2,442, against the winner's 4,028 and 2,806. Even the largest ratio measured, 3.83, leaves both ahead. The virtual best is another matter: it is the best of 28 solvers on each instance, and both arms fall behind its count once $k$ exceeds 1.43 (`satsuma`) or 1.49 (`hydra_circuit_satsuma`). `hydra_circuit_satsuma` solved 248 instances that no 2022 solver solved, `satsuma` 242, and each missed about 190 that some 2022 solver solved.

# 9 What the run says about the next stages

Five candidate stages were listed before the run with the anniversary instances they target, so that `satsuma`'s results would rank them (`doc/ideas_watchlist.md`, `tools/gbd/watchlists.py`). A stage pays only where `satsuma` fails, at 1.87 points per instance here.

- *Near-complete parity systems*, 21 instances: Tseitin, Urquhart and xor-chain formulas with one clause removed, so one parity is incomplete and the XOR stage hands on. `satsuma` failed all 21, as did all of 2022. Gaussian elimination over the complete parities with the incomplete one as clauses, or a case

split on its missing row, is the obvious stage; the symmetry probe explains why satsuma fails them, the missing clause kills the cycle flips.
- *Equivalence chains*, 10 instances (equilarge): `satsuma` failed all 10.
- *Algebraic multiplier miters*, 23 instances with explicit partial products: `satsuma` failed 16. The known method is algebraic (AMulet-style polynomial reduction with PAC certificates), a larger build.
- *Cook recognisers for more encodings*, 113 instances: `satsuma` failed 7, five relativised pigeonhole and two clique-colouring variants. Its symmetry breaking took the other 106, so this idea shrank from 56 unsolved-in-2022 targets to 7.
- *A bounded local-search stage*, 35 satisfiable instances a walk-heavy 2022 solver took quickly: `satsuma` failed 27, but a walk would rescue only some, at a cost on every instance.

The first two are the same mechanism as this run's unique wins, parity reasoning where symmetry breaking has nothing to break, and together they are 31 instances no entrant of 2022 or `satsuma` solves.

On the newer competition sets the density is higher: 4 unique wins in 391 instances on the 2026 main track, and a scan of the 2025 main track with the same probe predicts 4 to 5 of 400, three of them Tseitin grids of 124,500 to 319,200 variables on which satsuma's symmetry step finds a few dozen generators and the XOR stage finishes in 64 to 556 s (`doc/data/main_2025_stage_scan.md`).

# 10 Conclusion

On the broadest set the competition has, the structural stages cost 1.1 PAR-2 points and gained 17.6, almost all of it from eight instances no engine solves and the stages decide in milliseconds. The cost was 6% of the gain before the fix and will be about 4% after it. The stages did not lose a single instance, and their certificates are the same VeriPB and parity proofs that verified in the 2026 run. The limiting factor is the number of instances where structure is the only way in: `satsuma`'s symmetry breaking already takes the easy parity and pigeonhole formulas, which is exactly why the two families behind this run's eight wins, parity under cardinality and the largest chessboards, and the 31 near-complete parity and equivalence-chain instances still open, are the right next targets.

**Artifacts.** Run directory `sat_benchmarks/runs/anni_2022_duo` (results, per-job logs, stage scan, symmetry probe); reports in `doc/competition/` (`duo_anni_2022_5000_satsuma_vs_hydra_circuit_satsuma.md` and the two per-solver reports); the numbers and figures of this document from `doc/hydra_satsuma_2022_anniversary_figures.py`; the speed calibration in `doc/data/speed_calibration_sc2022.md`.

# References


[1] T. Balyo, M. J. H. Heule, M. Iser, M. Järvisalo, and M. Suda (eds.). Proceedings of SAT Competition 2022: Solver and Benchmark Descriptions. University of Helsinki, 2022. Results and detailed per-instance data: `https://satcompetition.github.io/2022/`.

[2] M. Anders, C. Codel, and M. J. H. Heule. Simplify, order, break, repeat. *SAT 2026*, LIPIcs vol. 377, article 4. satsuma: `https://github.com/markusa4/satsuma`; the SAT Competition 2026 winner `satsuma-iter+kissat`: `https://satcompetition.github.io/2026/downloads/solvers/anders.tar.xz`.

[3] S. A. Cook. A short proof of the pigeon hole principle using extended resolution. *SIGACT News*, 8(4):28–32, 1976.

[4] A. Haken. The intractability of resolution. *Theoretical Computer Science*, 39:297–308, 1985.

[5] S. Gocht and J. Nordström. Certifying parity reasoning efficiently using pseudo-Boolean proofs. *AAAI*, 2021.

[6] G. S. Tseitin. On the complexity of derivation in propositional calculus. *Studies in Constructive Mathematics and Mathematical Logic, Part II*, pp. 115–125, 1968.

[7] A. Urquhart. Hard examples for resolution. *Journal of the ACM*, 34(1):209–219, 1987.

[8] M. Soos, K. Nohl, and C. Castelluccia. Extending SAT solvers to cryptographic problems. *SAT*, LNCS 5584, pp. 244–257, 2009.